



SYMFONY CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & CONSOLE

Installation

```
composer create-project symfony/skeleton myapp
composer require symfony/webapp-pack
php bin/console server:start
php bin/console make:controller User
php bin/console doctrine:migrations:migrate
php bin/console debug:router
```

04. TWIG TEMPLATES

Workflow

```
return $this->render('user/index.html.twig', ...
{{ user.name }} {# comment #}
{% for u in users %}{{ u.name }}{% endfor %}
{% if user.admin %}Admin{% endif %}
{% extends 'base.html.twig' %}
{% block body %}...{% endblock %}
{{ path('user_list') }}
```

02. ROUTING

Architecture

```
// PHP Attribute style:
#[Route('/users', name: 'user_list', methods:...
public function index(): Response { ... }
// YAML (config/routes.yaml):
user_list:
  path: /users
  controller: App\Controller\UserController::in...
```

05. DOCTRINE ORM

CRUD API

```
#[ORM\Entity] #[ORM\Table(name: 'users')]
class User {
    #[ORM\Id] #[ORM\GeneratedValue]
    #[ORM\Column] private ?int $id = null;
    #[ORM\Column(length: 180, unique: true)]
    private string $email;
}
$em->persist($user); $em->flush();
$repo->find(1) / $repo->findBy(['active'=>tru...
```

03. CONTROLLERS

YAML / Config

```
class UserController extends AbstractControll...
#[Route('/users', methods: ['GET'])]
public function index(UserRepository $repo): ...
    return $this->json($repo->findAll());
}
#[Route('/users/{id}', methods: ['GET'])]
public function show(User $user): Response {
    return $this->json($user); // ParamConverter
}
}
```

06. SERVICES & DI

Integration

```
// Auto-wiring: constructor injection works a...
public function __construct(
    private readonly UserRepository $repo,
    private readonly MailerInterface $mailer
) {}
// config/services.yaml:
services: App\: resource: '../src/'
# Everything in src/ is auto-registered as a ...
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. FORMS

Hardening

```
$form = $this->createForm(UserType::class, $u...
$form->handleRequest($request);
if ($form->isSubmitted() && $form->isValid())...
$em->persist($user); $em->flush();
return $this->redirectToRoute('user_list');
}
return $this->render('user/edit.html.twig', [...
```

10. CONSOLE COMMANDS

Unit Tests

```
#[AsCommand(name: 'app:send-emails')]
class SendEmailsCommand extends Command {
protected function execute(InputInterface $in...
$out->writeln('Sending...');
return Command::SUCCESS;
}
}
php bin/console app:send-emails
```

08. VALIDATION

Observability

```
use Symfony\Component\Validator\Constraints a...
#[Assert\NotBlank]
#[Assert\Email]
private string $email;
$errors = $validator->validate($user);
if (count($errors) > 0) throw new ValidationF...
// Form validation is automatic via FormType
```

11. COMMON COMMANDS

Commands

```
php bin/console debug:container // list serv...
php bin/console debug:router // list rout...
php bin/console cache:clear
php bin/console make:entity/controller/form
php bin/console doctrine:database:create
php bin/console doctrine:migrations:diff // ...
```

09. SECURITY

Optimization

```
// security.yaml:
firewalls:
main:
stateless: true
jwt: ~ # LexikJWTAuthBundle
access_control:
- { path: ^/api/admin, roles: ROLE_ADMIN }
// In controller:
$this->denyAccessUnlessGranted('ROLE_ADMIN');
```

12. COMMON GOTCHAS

Warning

Services are private by default inject, don't fetch from container
Auto-wiring uses type-hints: same type = one service conflict
Doctrine: flush() is needed to persist to DB
ParamConverter: {id} param auto-loads entity by ID
Twig escaping: use |raw only for trusted HTML